Name: Grant Harvey Date: 10/29/2026

Honor Pledge: I pledge that the only resources I used to complete this homework were the course textbook, the course slides, or my personal course notes. I did not confer with any other student, consult any solutions manuals, printed or on-line, nor did I do any Internet searches to solve any problems for this homework assignment. The solutions below are my own work.

Student Signature: *Grant Harvey*

Cyber Security Fundamentals – CHAPTER 4 HOMEWORK

70 points total

4.0 [12] Use the Access Control System Commands Reference Sheet on the next page to answer the following question.

Rule	Command (by S_o)	Authorization	Operation
R1	**transfer** $\left\{ \begin{matrix} \alpha* \\ \alpha \end{matrix} \right\}$ **to** S, X	'α*' in $A[S_o, X]$	store $\left\{ \begin{matrix} \alpha* \\ \alpha \end{matrix} \right\}$ in $A[S, X]$
R2	**grant** $\left\{ \begin{matrix} \alpha* \\ \alpha \end{matrix} \right\}$ **to** S, X	'owner' in $A[S_o, X]$	store $\left\{ \begin{matrix} \alpha* \\ \alpha \end{matrix} \right\}$ in $A[S, X]$
R3	**delete** α **from** S, X	'control' in $A[S_o, S]$ or 'owner' in $A[S_o, X]$	delete α from $A[S, X]$
R4	$w \leftarrow$ **read** S, X	'control' in $A[S_o, S]$ or 'owner' in $A[S_o, X]$	copy $A[S, X]$ into w
R5	**create object** X	None	add column for X to A; store 'owner' in $A[S_o, X]$
R6	**destroy object** X	'owner' in $A[S_o, X]$	delete column for X from A
R7	**create subject** S	none	add row for S to A; execute **create object** S; store 'control' in $A[S, S]$
R8	**destroy subject** S	'owner' in $A[S_o, S]$	delete row for S from A; execute **destroy object** S

The four (4) commands below were issued by S_0 and are executed in the order indicated. Update **A** as indicated by the commands. The extra rows and columns as needed for any creates. For any **destroy** command, line through (if appropriate) rows/columns as needed.

transfer execute to S_2, O_3
grant execute* to S_1, O_0
create object O_4
destroy subject S_2

Access Control Matrix Problem

A

	O_0	O_1	O_2	O_3		S_0	S_1	S_2	
S_0	owner	owner	read*	execute		owner control	owner write	write	
S_1	read	read	owner			read	control	owner	
S_2		write	execute	owner		read		control	

Solution:

transfer execute to S_2, O_3

This transfer fails because S_0 only has execute on O_3 and does not have execute*

grant execute* to S_1, O_0

S0 is the owner of O_0, so it can grant execute* to S_1. S_1 already has read rights on O_0, so it adds execute* to it ending with read, execute*

create object O_4

Creates new object and sets the owner to be S_0

destroy subject S_2

S_0 does not own S_2, so it cannot destroy it, command failed

A — Updated

	O_0	O_1	O_2	O_3	O_4	S_0	S_1	S_2	
S_0	owner	owner	read*	execute	Owner	owner control	owner write	write	
S_1	Read, execute*	read	owner			read	control	owner	
S_2		write	execute	owner		read		control	

4.1 [15] For the DAC model, an alternative representation of the protection state is a directed graph. Each subject and each object in the protection state is represented by a node (a single node is used for an entity that is both subject and object). A directed line from a subject to an object indicates an access right, and the label on the link defines the access right.

a. Draw a directed graph that corresponds to the access matrix of Figure 4.2a.

		OBJECTS			
		File 1	File 2	File 3	File 4
SUBJECTS	User A	Own Read Write		Own Read Write	
	User B	Read	Own Read Write	Write	Read
	User C	Read Write	Read		Own Read Write

(a) Access matrix

b. Draw a directed graph that corresponds to the access matrix of Figure 4.3.

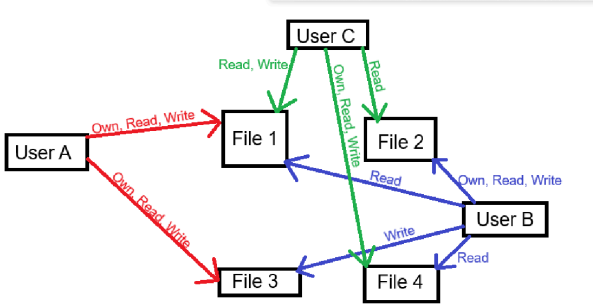
		OBJECTS								
		subjects			files		processes		disk drives	
		S_1	S_2	S_3	F_1	F_2	P_1	P_2	D_1	D_2
SUBJECTS	S_1	control	owner	owner control	read *	read owner	wakeup	wakeup	seek	owner
	S_2		control		write *	execute			owner	seek *
	S_3			control		write	stop			

* - copy flag set

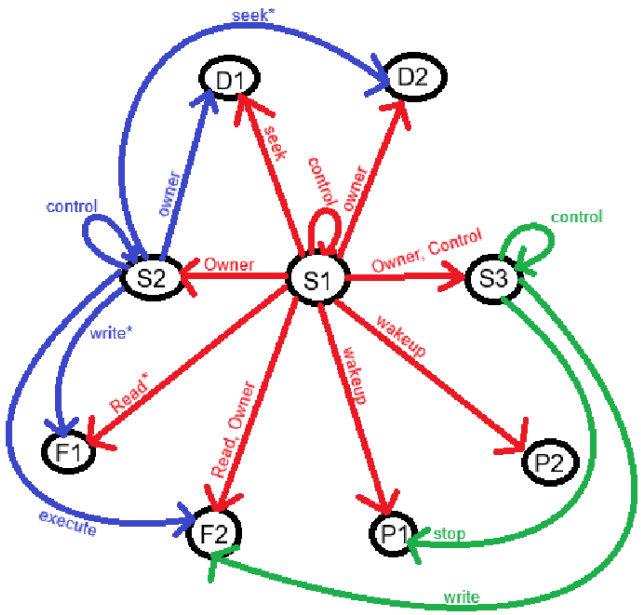
c. Is there a one-to-one correspondence between the directed graph representation and the access matrix representation? Explain.

Solution:

a. The colors are purely to differentiate users
Red = User A
Blue = User B
Green = User C



b. The colors are purely to differentiate users
Red = S1
Blue = S2
Green = S3



c.

Yes there is a 1 to 1 correspondence, anything in the table results in a directed line, then each table's contents are the line labels.

4.5 [14] UNIX treats file directories in the same fashion as files; that is, both are defined by the same type of data structure, called an inode. As with files, directories include a nine-bit protection string. If care is not taken, this can create access control problems. For example, consider a file with protection mode 644 (octal) contained in a directory with protection mode 730. How might the file be compromised in this case?

Solution:

- The files 9 bit string controls who can read or modify it.
- The directories 9 bit string controls who can open it, and who can add, delete, or rename the files inside.

Therefore in the UNIX environment described, if I have access to write in a directory, I also gain access to delete anything in that directory, even if I don't have specific write commands for a file. This means I could delete the file with protection code 644

4.6 [15] In the traditional UNIX file access model, UNIX systems provide a default setting for newly created files and directories, which the owner may later change. The default is typically full access for the owner combined with one of the following: (1) no access for group and other, (2) read/execute access for group and none for other, or (3) read/execute access for both group and other.

Briefly discuss the advantages and disadvantages of each of these cases, including an example of a type of organization where each would be appropriate.

Solution:
1. no access for group and other
Advantages:

- Maximum Privacy and Security, no one other than the creator can access the directory
- Minimizes the risk of exposure or unauthorized privilege abuse by other users

Disadvantages:

- No collaboration between users

Example:

- Defence contractors might use it because they deal with items of such high security they wouldn't accidentally want users without the right privilege to access files.

2. read/execute access for group and none for other
Advantages:

- Allows a group to seamlessly share files between users

Disadvantages:

- External users who might need files will need to be manually added

Example:

- Research Laboratories might use this structure so that everyone on the team can seamlessly access work or data from others while people outside the network cannot

3. read/execute access for both group and other
Advantages:

- Good for shared public archives or general resources that everyone needs to access easily

Disadvantages:

- Very high security risk; any user on the system can read, copy, or execute the files/scripts

Example:

- Educational institutions might use it for shared computers in each class room, that way one computer can store all the class slides and stuff and every teacher / student can access them
- Also might be useful for open-source code repositories

4.8 [14] Assume a system with N job positions. For job position i, the number of individual users in that position is U_i and the number of permissions required for the job position is P_i.

a. For a traditional DAC scheme, how many relationships between users and permissions must be defined?

b. For a RBAC scheme, how many relationships between users and permissions must be defined?

Solution:

a.

You need P_i permissions for each U_i in a job position. So for each job position you need (P_i * U_i) relationships between users and permissions. Now do that for each job position and add them all together. Something like this:

Foreach i in N:
    total += U_i * P_i
return total

b.

In a RBAC, all permissions are assigned to roles rather than users with each job positon being a role. Then we also have the relationship between users and their role. So each job position, i, gets P_i relationships for permissions, and U_i relationships for users being assigned a role. Something like this:

Foreach i in N:
    total += U_i + P_i
return total